

Parallel and Distributed Programming Analysis

Josh Zrubek, Henry Hornung, John Spexarth

May 2026

Software Architecture

Overview

This document will evaluate three separate programs, each using three different methods of parallelization—`pthread`, `openmp`, `mpi`—as the number of nodes, number of threads, and amount of memory varies.

Each implementation has been tested in batches of ten, measuring their runtime in seconds, maximum memory usage in kilobytes, and their average cpu utilization as a percentage.

File I/O

The `pthread` and `openmp` implementations read in the wiki dump text file differently than the `mpi` implementation due to some architectural decisions.

The `pthread` and `openmp` implementation read the wiki dump text file in 20,000 lines at a time. With an average character count per line of 1,700, this amounts to approximately $\frac{20,000 \cdot 1,700}{1,048,576} = 32.42$ MB. Notably, this size fits comfortably in the smallest amount of memory we test.

The `mpi` implementation reads the entire wiki dump text file in at once. Since the file is approximately 1.7 GB, it will need at least that much memory to not run out.

Each program uses a `file_string_array_t` struct, which stores the lines of the wiki dump text file as a `**char`.

Pthreads

This implementation uses a `process_chunk` function to spawn each `pthread` at for each chunk read in from the wiki dump text file. This may contribute to a hit in performance due to the overhead of having to create threads for each chunk. Each `pthread` modifies a global `max_values` integer array. No locks are needed because each `pthread` has a mutually exclusive range of lines with each index of `max_values` corresponding to a line.

Openmp

This implementation uses a single for loop and the `#pragma omp parallel for` preprocessor directive to parallelize the task. Each thread modifies a global `max_values` integer array at a global `max_values_i` index plus their local index inside the for loop. This avoids a race condition because each thread will have a mutually exclusive range of `i` in the for loop.

Mpi

This implementation uses the main process (rank 0) to load the text file into memory. Then, the `file_string_array` structure of the main process divides up the lines amongst all threads, and sends them to other threads using `MPI.Send`. Doing so keeps the threads' memory separate, avoiding a race condition, and each thread only allocates memory it will use. The max ascii values of each line in the file are sent back to an array within the main thread's memory, which is then processed into the program's output.

Performance Analysis

Pthreads Results

The following table summarizes the collected performance data for each batch given their nodes, threads, and memory sizes.

Nodes	Threads	Memory	Runtime Mean	Max Memory Mean	Cpu Mean
1	1	128M	9.4	39337.6	99.0
1	1	64M	9.8	39376.8	92.4
1	2	128M	5.4	39315.6	172.4
1	2	64M	5.6	39336.8	166.6
1	4	128M	3.6	39133.2	280.4
1	4	64M	3.6	39122.4	276.2
1	8	128M	2.5	38822.8	424.8
1	8	64M	2.5	38823.2	428.0
1	16	64M	2.4	38501.6	448.9
2	1	128M	9.1	39362.0	99.0
2	1	64M	9.5	39348.4	94.3
2	2	128M	5.3	39327.6	173.0
2	2	64M	5.6	39380.0	166.8
2	4	128M	3.5	39151.6	283.0
2	4	64M	3.5	39100.0	279.5
2	8	128M	2.9	38859.6	347.1
2	8	64M	2.4	38816.4	428.4

Nodes	Threads	Memory	Runtime Std	Max Memory Std	Cpu Std
1	1	128M	0.26618	27.20784	0.00000
1	1	64M	0.80835	58.72119	6.88315
1	2	128M	0.04581	74.07984	1.57762
1	2	64M	0.59350	86.17914	14.02537
1	4	128M	0.09466	143.55549	0.96609
1	4	64M	0.18445	161.01014	12.44365
1	8	128M	0.12675	92.41549	14.40525
1	8	64M	0.07288	76.77500	8.81917
1	16	64M	0.01252	288.26038	1.91195
2	1	128M	0.01713	51.16422	0.00000
2	1	64M	0.52101	45.93522	5.03433
2	2	128M	0.06721	62.35240	2.16025
2	2	64M	0.57291	55.45769	13.71779
2	4	128M	0.00422	109.81115	0.00000
2	4	64M	0.01578	99.25948	0.97183
2	8	128M	0.02309	124.94194	3.44642
2	8	64M	0.00949	131.55413	1.34990

Runtime Analysis

Figure 1 shows the mean runtime for each configuration, with error bars representing one standard deviation.

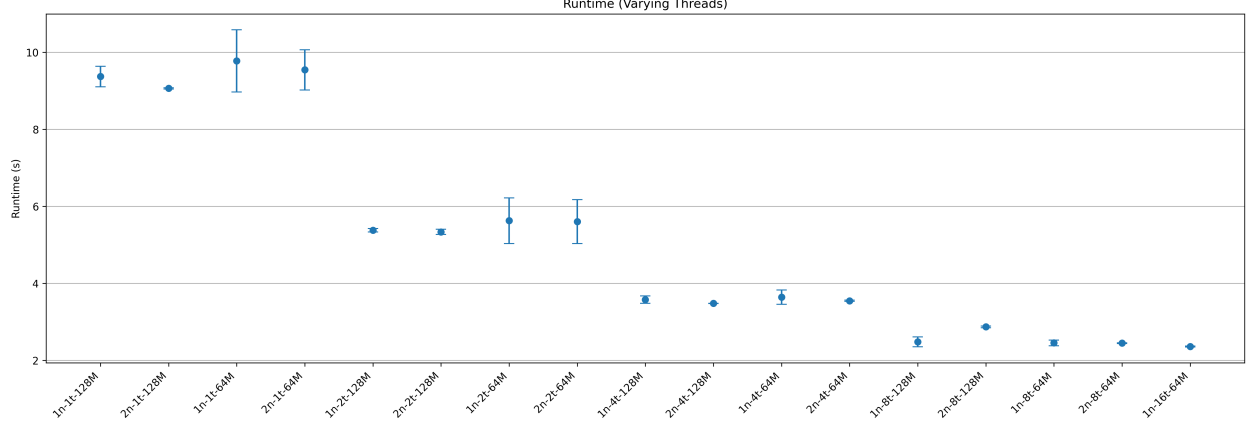


Figure 1: Mean runtime across each batch

To better understand the effect of threading, Figure 1 isolates configurations where only the number of threads varies while other parameters remain constant.

It can be observed that increasing the number of threads results in a substantial decrease in mean runtime. It does appear as though this relationship is not linear, however, as the batch with 4 threads is not as far from the batch with 2 threads as the batch with 2 threads is from the batch with 1 thread.

When holding the number of threads and nodes constant, increasing memory from 64 MB to 128 MB does not significantly impact runtime. Similarly, increasing the number of nodes from 1 to 2 does not significantly

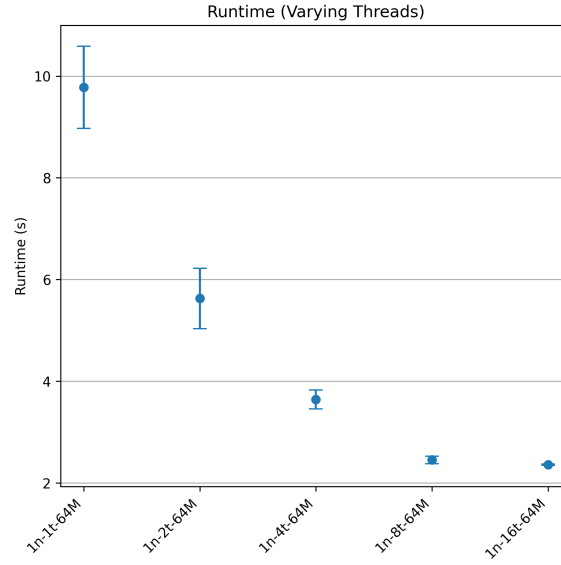


Figure 2: Effect of increasing thread count on runtime

impact runtime.

CPU Utilization Analysis

Figure 3 illustrates CPU utilization across configurations.

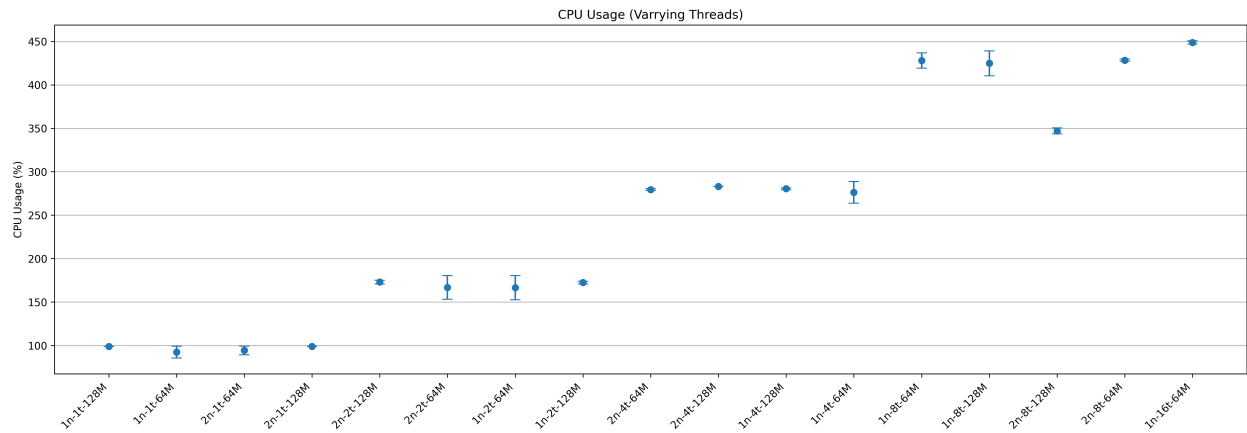


Figure 3: CPU utilization across configurations

It can be observed that CPU utilization increases with the number of threads, exceeding 100% in multi-threaded runs. This is expected behavior, as the program we used to track CPU usage, time, aggregates it across cores.

OpenMP Results

The following tables summarize the collected performance data for each batch given their nodes, threads, and memory sizes.

Nodes	Threads	Memory	Runtime Mean	Max Memory Mean	Cpu Mean
1	2	1G	6.0	39536.4	177.0
1	2	256M	6.0	39539.6	176.9
1	2	512M	6.0	39493.2	177.0
1	4	1G	3.8	39401.2	288.1
1	4	2G	3.7	39347.2	291.9
1	4	512M	3.7	39324.4	292.0

Nodes	Threads	Memory	Runtime Std	Max Memory Std	Cpu Std
1	2	1G	0.00738	64.15121	0.00000
1	2	256M	0.00738	45.62455	0.31623
1	2	512M	0.01033	58.66818	0.00000
1	4	1G	0.03596	86.59587	2.60128
1	4	2G	0.01287	98.82510	0.73786
1	4	512M	0.00843	92.76877	0.47140

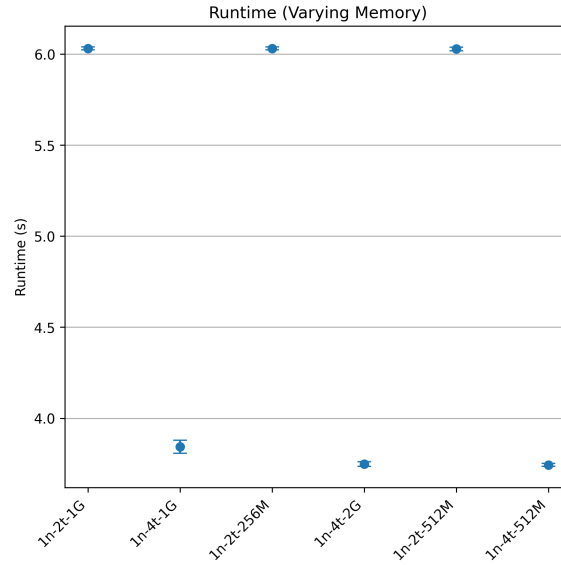


Figure 4: Runtime across batches

0.0.1 Runtime Analysis

As seen in Figure 4, as the number of threads increases, the runtime drops significantly. Furthermore, the standard deviation for every batch is incredibly small.

Additionally, in Figure 4, as memory increases barely any effect is observed on runtime.

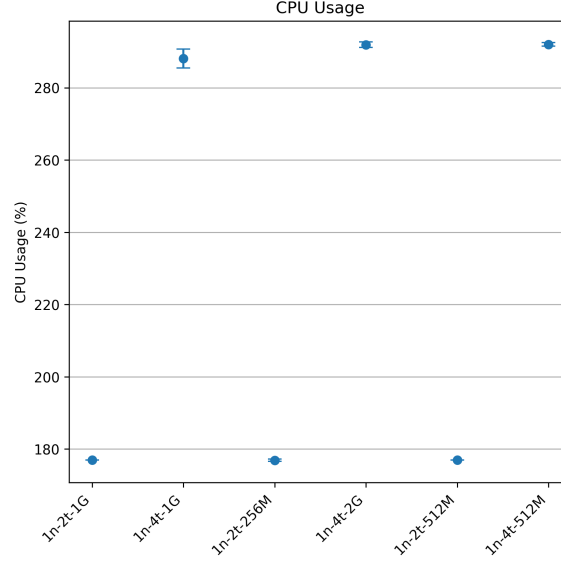


Figure 5: CPU utilization across batches

0.0.2 CPU Utilization Analysis

It can be observed that, as we expect, in Figure 5 the more threads that are used, the higher the CPU usage.

MPI Results

The following tables summarize the collected performance data for each batch given their nodes, threads, and memory sizes.

Nodes	Threads	Memory	Runtime Mean	Max Memory Mean	Cpu Mean
1	1	3G	17.9	1718311.6	92.6
1	2	3G	17.7	1853492.8	180.7
2	1	3G	25.1	1717386.4	91.9
2	2	3G	22.6	1853034.0	178.2
2	4	3G	19.4	1849881.6	331.8
4	2	3G	21.5	1853047.6	177.2
4	4	3G	21.2	1750692.4	343.0

Nodes	Threads	Memory	Runtime Std	Max Memory Std	Cpu Std
1	1	3G	1.40932	177.35288	0.96609
1	2	3G	1.17823	114.19749	3.05687
2	1	3G	3.09101	669.48259	2.68535
2	2	3G	2.72001	622.19754	6.92499
2	4	3G	0.77786	376.65758	12.87375
4	2	3G	0.71662	889.17930	6.03324
4	4	3G	0.58961	2835.59624	9.28559

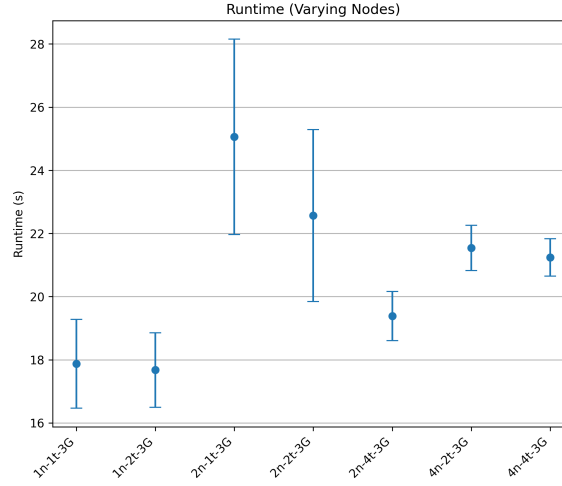


Figure 6: CPU utilization across batches

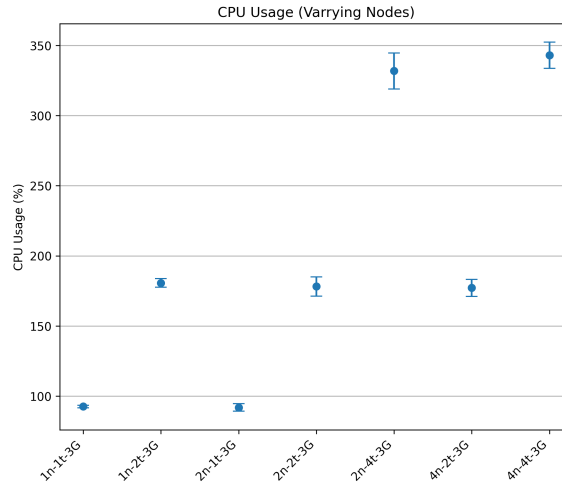


Figure 7: CPU utilization across batches

Runtime Analysis

In Figure 6, we can observe that, interestingly, as the number of threads or nodes increases, there is a jump in runtime. There appears to be a decrease in runtime from 2 to 4 nodes and/or threads, however, these points do not appear to be statistically significant.

CPU Usage Analysis

In Figure 7, we can observe that as the number of nodes and threads increase, the CPU utilization increases significantly.

A First 100 Lines of Output

Line: 0; Max ASCII: 125
Line: 1; Max ASCII: 125
Line: 2; Max ASCII: 125
Line: 3; Max ASCII: 125
Line: 4; Max ASCII: 125
Line: 5; Max ASCII: 125
Line: 6; Max ASCII: 125
Line: 7; Max ASCII: 125
Line: 8; Max ASCII: 125
Line: 9; Max ASCII: 124
Line: 10; Max ASCII: 125
Line: 11; Max ASCII: 125
Line: 12; Max ASCII: 125
Line: 13; Max ASCII: 126
Line: 14; Max ASCII: 125
Line: 15; Max ASCII: 125
Line: 16; Max ASCII: 125
Line: 17; Max ASCII: 125
Line: 18; Max ASCII: 125
Line: 19; Max ASCII: 125
Line: 20; Max ASCII: 125
Line: 21; Max ASCII: 125
Line: 22; Max ASCII: 125
Line: 23; Max ASCII: 125
Line: 24; Max ASCII: 125
Line: 25; Max ASCII: 124
Line: 26; Max ASCII: 125
Line: 27; Max ASCII: 125
Line: 28; Max ASCII: 125
Line: 29; Max ASCII: 125
Line: 30; Max ASCII: 125
Line: 31; Max ASCII: 125
Line: 32; Max ASCII: 125
Line: 33; Max ASCII: 125
Line: 34; Max ASCII: 125
Line: 35; Max ASCII: 125
Line: 36; Max ASCII: 125
Line: 37; Max ASCII: 125
Line: 38; Max ASCII: 125
Line: 39; Max ASCII: 125
Line: 40; Max ASCII: 125
Line: 41; Max ASCII: 125
Line: 42; Max ASCII: 125
Line: 43; Max ASCII: 125
Line: 44; Max ASCII: 125
Line: 45; Max ASCII: 125
Line: 46; Max ASCII: 125
Line: 47; Max ASCII: 125
Line: 48; Max ASCII: 125

Line: 49; Max ASCII: 125
Line: 50; Max ASCII: 125
Line: 51; Max ASCII: 125
Line: 52; Max ASCII: 122
Line: 53; Max ASCII: 125
Line: 54; Max ASCII: 125
Line: 55; Max ASCII: 125
Line: 56; Max ASCII: 125
Line: 57; Max ASCII: 125
Line: 58; Max ASCII: 125
Line: 59; Max ASCII: 125
Line: 60; Max ASCII: 125
Line: 61; Max ASCII: 125
Line: 62; Max ASCII: 125
Line: 63; Max ASCII: 125
Line: 64; Max ASCII: 125
Line: 65; Max ASCII: 125
Line: 66; Max ASCII: 125
Line: 67; Max ASCII: 125
Line: 68; Max ASCII: 125
Line: 69; Max ASCII: 125
Line: 70; Max ASCII: 124
Line: 71; Max ASCII: 125
Line: 72; Max ASCII: 125
Line: 73; Max ASCII: 125
Line: 74; Max ASCII: 125
Line: 75; Max ASCII: 125
Line: 76; Max ASCII: 125
Line: 77; Max ASCII: 125
Line: 78; Max ASCII: 125
Line: 79; Max ASCII: 125
Line: 80; Max ASCII: 125
Line: 81; Max ASCII: 125
Line: 82; Max ASCII: 125
Line: 83; Max ASCII: 125
Line: 84; Max ASCII: 125
Line: 85; Max ASCII: 125
Line: 86; Max ASCII: 125
Line: 87; Max ASCII: 125
Line: 88; Max ASCII: 125
Line: 89; Max ASCII: 125
Line: 90; Max ASCII: 125
Line: 91; Max ASCII: 125
Line: 92; Max ASCII: 125
Line: 93; Max ASCII: 125
Line: 94; Max ASCII: 125
Line: 95; Max ASCII: 125
Line: 96; Max ASCII: 125
Line: 97; Max ASCII: 122
Line: 98; Max ASCII: 125
Line: 99; Max ASCII: 125

B Batch Scripts

B.1 pthread

```
#!/bin/bash
## Script for running project 4 in Operating System - OpenMP

## SBATCH variables
#SBATCH --job-name=openmp-test
#SBATCH --nodes=2
#SBATCH --cpus-per-task=16
#SBATCH --mem-per-cpu=128M
#SBATCH --time=00:05:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=jazrubek@ksu.edu
#SBATCH --constraint=moles
#SBATCH --partition=killable.q

## Program variables
NODES=2
THREADS=16
MEMORY="128M"
PROGRAM_NAME="3way-pthread"
PROGRAM_PATH="${HOME}/cis520-project-4-parallel-and-distributed-programming/${PROGRAM_NAME}/build/${PROGRAM_NAME}"
LOG_FILENAME="${NODES}x${THREADS}x${MEMORY}_${PROGRAM_NAME}_log.txt"
CSV_FILENAME="${NODES}x${THREADS}x${MEMORY}_${PROGRAM_NAME}.csv"
OUTPUT_FILENAME="${NODES}x${THREADS}x${MEMORY}_${PROGRAM_NAME}_output.txt"

cp $PROGRAM_PATH $TMPDIR
pushd $TMPDIR

echo "Running ${PROGRAM_NAME} with ${MEMORY} memory, ${THREADS} thread(s), and ${NODES} node(s)" >> $LOG_FILENAME

echo "Runtime (s),Max Memory (KB),CPU Usage (%)" >> $CSV_FILENAME

for i in {0..9}; do
    /usr/bin/time --format="%e,%M,%P" ./$PROGRAM_NAME $THREADS >> $OUTPUT_FILENAME
    2>> $CSV_FILENAME
done

kstat --table-cpu-usage-1min "$SLURM_JOB_ID" >> $LOG_FILENAME

popd
mkdir -p $HOME/$PROGRAM_NAME/logs/
mkdir -p $HOME/$PROGRAM_NAME/output/
mkdir -p $HOME/$PROGRAM_NAME/csv/
cp $TMPDIR/$OUTPUT_FILENAME $HOME/$PROGRAM_NAME/output/$OUTPUT_FILENAME
cp $TMPDIR/$LOG_FILENAME $HOME/$PROGRAM_NAME/logs/$LOG_FILENAME
```

```
cp $TMPDIR/$CSV_FILENAME $HOME/$PROGRAM_NAME/csv/$CSV_FILENAME
```

B.2 openmp

```
#!/bin/bash
## Script for running project 4 in Operating System - OpenMP

## SBATCH variables
#SBATCH --job-name=openmp-test
#SBATCH --nodes=2
#SBATCH --cpus-per-task=16
#SBATCH --mem-per-cpu=128M
#SBATCH --time=00:05:00
#SBATCH --mail-type=ALL
#SBATCH --mail-user=jazrubek@ksu.edu
#SBATCH --constraint=moles
#SBATCH --partition=killable.q

## Program variables
NODES=2
THREADS=16
MEMORY="128M"
PROGRAM_NAME="3way-openmp"
PROGRAM_PATH="${HOME}/cis520-project-4-parallel-and-distributed-programming/${
    PROGRAM_NAME}/build/${PROGRAM_NAME}"
LOG_FILENAME="${NODES}x${THREADS}x${MEMORY}_${PROGRAM_NAME}_log.txt"
CSV_FILENAME="${NODES}x${THREADS}x${MEMORY}_${PROGRAM_NAME}.csv"
OUTPUT_FILENAME="${NODES}x${THREADS}x${MEMORY}_${PROGRAM_NAME}_output.txt"

cp $PROGRAM_PATH $TMPDIR
pushd $TMPDIR

echo "Running ${PROGRAM_NAME} with ${MEMORY} memory, ${THREADS} thread(s), and ${
    NODES} node(s)" >> $LOG_FILENAME

echo "Runtime (s),Max Memory (KB),CPU Usage (%)" >> $CSV_FILENAME

for i in {0..9}; do
    /usr/bin/time --format="%e,%M,%P" ./$PROGRAM_NAME $THREADS >> $OUTPUT_FILENAME
    2>> $CSV_FILENAME
done

kstat --table-cpu-usage-1min "$SLURM_JOB_ID" >> $LOG_FILENAME

popd
mkdir -p $HOME/$PROGRAM_NAME/logs/
mkdir -p $HOME/$PROGRAM_NAME/output/
mkdir -p $HOME/$PROGRAM_NAME/csv/
cp $TMPDIR/$OUTPUT_FILENAME $HOME/$PROGRAM_NAME/output/$OUTPUT_FILENAME
cp $TMPDIR/$LOG_FILENAME $HOME/$PROGRAM_NAME/logs/$LOG_FILENAME
```

```
cp $TMPDIR/$CSV_FILENAME $HOME/$PROGRAM_NAME/csv/$CSV_FILENAME
```

B.3 mpi

```
#!/bin/bash

## SBATCH Variables
#SBATCH --job-name=mpi-testing
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=4
#SBATCH --mem-per-cpu=3G
#SBATCH --time=00:05:00
#SBATCH --partition=killable.q
#SBATCH --constraint=moles
#SBATCH --mail-type=ALL
#SBATCH --mail-user=jazrubek@ksu.edu

## Program Variables
PROGRAM_NAME="3way-mpi"
NODES=2
THREADS=4
MEMORY="3G"

## Output Files
PROGRAM_OUTPUT="${HOME}/${PROGRAM_NAME}/output/${NODES}x${THREADS}x${MEMORY}_${PROGRAM_NAME}_output.txt"
PROGRAM_LOGS="${HOME}/${PROGRAM_NAME}/logs/${NODES}x${THREADS}x${MEMORY}_${PROGRAM_NAME}_logs.txt"
CSV_FILENAME="${HOME}/${PROGRAM_NAME}/csv/${NODES}x${THREADS}x${MEMORY}_${PROGRAM_NAME}.csv"

## Create Output Directories
mkdir -p $HOME/$PROGRAM_NAME/output
mkdir -p $HOME/$PROGRAM_NAME/logs
mkdir -p $HOME/$PROGRAM_NAME/csv

## Load Modules
module reset
module load CMake/3.23.1-GCCcore-11.3.0 foss/2022a OpenMPI/4.1.4-GCC-11.3.0

echo "Runtime (s), Max Memory (KB), CPU Usage (%)" > $CSV_FILENAME

for i in {0..9}; do
    /usr/bin/time --format="%e,%M,%P" mpirun $HOME/cis520-project-4-parallel-
    and-distributed-programming/$PROGRAM_NAME/build/$PROGRAM_NAME >
    $PROGRAM_OUTPUT 2>> $CSV_FILENAME
done
```